Assignment-2
Part I

Q1.

In SQL, NULL represents an unknown or missing value. Any comparison using the = operator against null evaluates to UNKNOWN. Since the WHERE clause only passes rows that evaluate to TRUE, no rows are returned, and COUNT(*) yields 0 regardless of how many students actually lack a phone number.

Corrected Query: SELECT COUNT (*) FROM WHERE Student phone__number IS NULL;

Aggregate functions such as COUNT(column), SUM, AVG, MIN, and MAX silently skip NULL values because including an unknown quantity in a calculation would make the result meaningless or undefined.

Q2.

The student is correct

DISTINCT is redundant here. When GROUP BY department is present, the query already produces exactly one output row per unique value of department . DISTINCT would attempt to eliminate duplicate (department, count) pairs from the result set, but since each department appears exactly once (guaranteed by GROUP BY), there are no duplicates to remove. The keyword does no additional work and only adds visual noise.

Q3.

The LEFT JOIN returns more rows than the INNER JOIN, which means there exist customers in the Customer table who have no matching rows in the Order table — i.e., customers who have never placed an order. The INNER JOIN eliminates these customers because it only retains rows where a match exists in both tables.

Suppose Customer has 500 rows and Order references only 380 distinct customers. The INNER JOIN returns 380 rows (one per matched customer, assuming one order each for simplicity). The LEFT JOIN returns all 500 Customer rows — the 380 with order data filled in and the remaining 120 with NULL in all Order columns, since those customers never ordered.

Q4.

In standard SQL, every column in the SELECT list must either appear in the GROUP BY clause or be wrapped in an aggregate function. Here, employee_name is neither grouped by nor aggregated.

After grouping by department, each group contains multiple employees. There is no single, unambiguous value of employee_name to display for the group. SQL would have to arbitrarily pick one employee name from each group - a non-deterministic choice that produces misleading results (the selected name would have no guaranteed relationship to the AVG(salary) shown). Some databases (e.g., older MySQL with sql_mode='') permit this but return an unpredictable name.

Corrected Query : select department, AVG(salary) from Employee group by department ;

Q5.

Error - AVG(salary) is an aggregate function and cannot appear in a WHERE clause.

SQL processes follow the logical order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

The WHERE clause filters individual rows before any grouping takes place — at that stage, groups have not yet been formed and aggregate values do not yet exist.

Corrected query: select department , AVG(salary) from Employee group by department having AVG(salary) >80000;

Q6.

(a)

Products priced above the overall average → Non-correlated subquery. The overall average price is a single scalar value computed once across all products — it does not depend on any particular row of the outer query. A non-correlated subquery computes this value once and reuses it, making it efficient.

SELECT * FROM Product WHERE price > (SELECT AVG(price) FROM Product);

(b)

Employees earning more than their own department's average → Correlated subquery

The average changes for every employee depending on which department they belong to. The subquery must be re-evaluated for each row of the outer query, referencing the outer row's department — making it correlated.

SELECT e1.employee_name, e1.department, e1.salary FROM Employee e1 WHERE e1.salary > (SELECT AVG(e2.salary) FROM Employee e2 WHERE e2.department = e1.department);

A non-correlated subquery executes once, and its result is reused for every row of the outer query. A correlated subquery re-executes once per row of the outer table — so if the outer table has N rows, the subquery runs N times. When the outer table is large, this becomes O(N) executions, which can be significantly slower. In practice, correlated subqueries on large datasets are often rewritten as JOINs with aggregations for better performance.

Part II:

(a)

```
SELECT c.customer_id, c.name, c.email, c.phone FROM Customer c LEFT JOIN
"Order" o ON c.customer_id = o.customer_id WHERE o.order_id IS NULL;
```

Justification: LEFT JOIN keeps all customers; WHERE o.order_id IS NULL isolates those with no matching order. An INNER JOIN would silently drop them.

(b)

```
SELECT o.restaurant_id, r.name AS restaurant_name, AVG(order_total) AS
avg_order_value FROM ( SELECT oi.order_id, o.restaurant_id, SUM(mi.price *
oi.quantity) AS order_total FROM OrderItem oi JOIN "Order" o ON oi.order_id = o.order_id
JOIN MenuItem mi ON oi.item_id = mi.item_id GROUP BY oi.order_id, o.restaurant_id )
AS order_totals JOIN Restaurant r ON
order_totals.restaurant_id = r.restaurant_id GROUP BY o.restaurant_id, r.name HAVING
COUNT(*) > 5;
```

Justification: The inner subquery computes each order's total first; the outer query then averages those totals per restaurant. HAVING (not WHERE) is used because the row-count filter applies after grouping.

(c)

```
WITH order_totals AS ( SELECT oi.order_id, o.restaurant_id, SUM(mi.price * oi.quantity) AS
order_total FROM OrderItem oi JOIN "Order" o ON oi.order_id = o.order_id JOIN MenuItem
mi ON oi.item_id = mi.item_id GROUP BY oi.order_id, o.restaurant_id ), restaurant_avg AS (
SELECT restaurant_id, AVG(order_total) AS avg_order_value, COUNT(*) AS num_orders
FROM order_totals GROUP BY restaurant_id HAVING COUNT(*) > 5 ) SELECT
ra.restaurant_id, r.name, ra.avg_order_value FROM restaurant_avg ra JOIN Restaurant r ON
ra.restaurant_id = r.restaurant_id WHERE ra.avg_order_value > ( SELECT
AVG(order_total) FROM order_totals );
```

Justification:

The platform-wide average is a single constant — a non-correlated scalar subquery computes it once and reuses it for every row. A correlated subquery would re-execute once per restaurant, which is unnecessary.

(d)

```
SELECT mi.restaurant_id, oi.item_id, mi.item_name, SUM(oi.quantity) AS total_quantity
FROM OrderItem oi JOIN MenuItem mi ON oi.item_id = mi.item_id GROUP BY
mi.restaurant_id, oi.item_id, mi.item_name HAVING SUM(oi.quantity) = ( SELECT
MAX(sub.total_qty) FROM ( SELECT oi2.item_id, SUM(oi2.quantity) AS total_qty FROM
OrderItem oi2 JOIN MenuItem mi2 ON oi2.item_id = mi2.item_id WHERE
mi2.restaurant_id = mi.restaurant_id GROUP BY oi2.item_id ) sub );
```

The correlated subquery finds the max total quantity for items in the same restaurant; HAVING keeps only items matching that max (correctly returning all tied items).

Limitation of GROUP BY + MAX for top-N per group: MAX() returns only the maximum value, not the full row — so you must re-join to identify which item holds that value. In case of ties, this either returns multiple rows or misses some, depending on the query. For reliable top-N per group (especially N > 1), window functions (RANK() / ROW_NUMBER() OVER PARTITION BY) are the correct approach.

Part III

Q1.

- student_id → student_name
- (student_id, course_id) → grade
- (student_id, course_id) → enrollment_date
- course_id → course_name
- course_id → instructor_id
- instructor_id → instructor_name
- instructor_id → instructor_office

Q2.

Yes. Every attribute holds a single atomic value, there are no repeating groups, and a primary key (student_id,course_id) exists. 1NF is satisfied.

Q3.

Multiple partial dependencies exist since some attributes depend on only *part* of the composite key:

Student_id - student_name (only needs student_id)

course_id → course_name, instructor_id, instructor_name, instructor_office (only needs course_id)

Concrete anomaly example — Update anomaly:
If a student's name changes, it must be updated in *every* row where that student appears. Missing even one row creates inconsistency (two different names for the same `student_id`).

Insertion anomaly**:** A new student can't be recorded until they enroll in a course, since `course_id` is part of the PK and can't be null.

Deletion anomaly: Dropping a student's last enrollment deletes their name from the database entirely.

Q4.

course_id → instructor_id → instructor_name, instructor_office

instructor_name and instructor_office depend on instructor_id, not directly on the primary key. This is a transitive dependency through a non-key attribute.

Anomaly: If an instructor's office changes, every course row for that instructor must be updated. If a course is deleted, the instructor's office information is lost entirely.

Q5.

Student(student_id (PK), student_name)

Justification: student_id → student_name

Course(course_id (PK), course_name, instructor_id (FK))

Justification: course_id → course_name; course_id → instructor_id

Instructor(instructor_id (PK), instructor_name, instructor_office)

Justification: instructor_id → instructor_name; instructor_id → instructor_office

Enrollment(student_id (FK, PK), course_id (FK, PK), grade, enrollment_date)

Justification: (student_id, course_id) → grade; (student_id, course_id) → enrollment_date

Part IV

JOIN vs. Subquery Choice

For finding customers who never placed an order, I could have used NOT IN with a subquery instead of a LEFT JOIN. However, NOT IN behaves incorrectly if the subquery returns even one NULL — it causes the entire outer query to return no rows, a silent and dangerous bug. The LEFT JOIN with WHERE o.order_id IS NULL is NULL-safe and equally readable, so it was the clear choice.

Normalization Trade-offs: 3NF vs. Denormalization

once the Enrollment table is split into Student, Course, Instructor, and Enrollment, a query like "list all students with their course name and instructor's office" now requires four-table JOINs instead of a simple SELECT from one table. This increases query complexity and can hurt read performance on large datasets without proper indexing.

Real systems often tolerate some denormalization for this reason. Data warehouses and reporting systems commonly use denormalized schemas (like star schemas) to avoid expensive joins at query time. Even in transactional systems, teams sometimes maintain a denormalized read-optimized table alongside the normalized schema — accepting some redundancy in exchange for query speed and simplicity.